The Journal of Open Source Software

PyTDLM: A Python framework for the systematic comparison of trip distribution laws and models

Rémi Perrier [1], **Floriana Gargiulo** [1], **Cyril Jayet** [1], and **Maxime Lenormand** [2]

1 Sorbonne Université, CNRS, Groupe d'Étude des Méthodes de l'Analyse Sociologique de la Sorbonne, GEMASS, Paris, France **2** TETIS, Univ Montpellier, AgroParisTech, CIRAD, CNRS, INRAE, Montpellier, France

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 21 November 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Spatial interaction models provide a quantitative description of how individuals, goods, or information move between locations. In transportation research and urban geography, these models are used to estimate **trip distribution**, the step of the classical four-stage transport modelling framework that allocates trip origins to trip destinations through an Origin–Destination (OD) matrix (Barbosa et al., 2018; Lenormand et al., 2016). A variety of laws, such as gravity-type decay functions or intervening-opportunities mechanisms, and several modelling strategies have been proposed, but rigorous comparison is challenging when law and model components are tightly coupled.

The *TDLM* framework was originally introduced to enable fair comparisons of trip distribution laws and models through a two-step procedure separating (i) the probability law governing the interaction process and (ii) the constrained model generating OD flows from that law. An implementation in R was published to facilitate adoption of this methodology (Lenormand, 2023).

Built on NumPy and SciPy, **PyTDLM** provides a full, native Python implementation of this framework, complementing the original Java-backed R codebase and extending its capabilities. In addition to porting the core methodology, the package offers vectorized algorithms, parallel execution, and introduces new functionality that simplifies calibration workflows and improves computational performance, making the framework accessible to researchers working within the Python scientific ecosystem.

Statement of need

Most trip distribution models combine two distinct mechanisms: a *law* that specifies how interaction probability decreases or increases with distance or opportunities, and a *model* that constrains flows according to marginal totals. When these components are not explicitly separated, comparisons between gravity-based and intervening-opportunities-based approaches can lead to misleading conclusions (Lenormand et al., 2012; Masucci et al., 2013; Simini et al., 2012; Yang et al., 2014).

Several R packages offer implementations of spatial interaction models, including **gravity** (Wölwer et al., 2018), **spflow** (Dargel & Laurent, 2021), **mobility** (Giles, 2021), and **simodels** (Lovelace & Nowosad, 2023). While valuable, these tools either integrate the law and model components in ways that hinder systematic comparison, or they do not provide built-in evaluation tools for contrasting observed and simulated OD matrices. This observation also applies to the Python ecosystem, where libraries such as **scikit-mobility** (Pappalardo &

Pellungrini, 2022) or **SpInt** (Oshan, 2016) present the same caveats. The TDLM methodology addresses these limitations by making the decomposition of mechanisms explicit.

The **PyTDLM** package responds to the need for a Python-native implementation that both reproduces the TDLM methodology and expands it to support high-performance computation and simplified calibration, while enabling integration with libraries such as NumPy, SciPy, Pandas, and Matplotlib.

Functionality

PyTDLM is available on PyPI, conda-forge and GitHub. Documentation and a tutorial using United States commuting data are available on the package website.

The package creates a pipeline for generating and validating OD matrices:

- **`run_law_model_gof`** A new high-level function introduced in the Python version, designed for efficiency. It computes mobility flows and goodness-of-fit metrics in a single step. By computing goodness-of-fit metrics on the fly without persisting intermediate simulated matrices, it significantly reduces memory overhead compared to the traditional stepwise approach. When several exponents are provided, computations are automatically dispatched to multiple threads.
- **`run_optimization`** A major addition to the Python port, this function automates parameter calibration. This function wraps `scipy.optimize.minimize_scalar` to determine the exponent that best maximizes or minimizes a selected goodness-of-fit measure. When multiple realizations are requested, the function parallelizes the computation of realizations and passes the averaged metric to the optimizer.
- **Core Components (`run_law`, `run_model`, `run_law_model`)**: These functions provide granular access to the two-step generation process and now also supports multi-exponent parallelization:
  - `run_law`: Computes probability matrices based on spatial distribution laws (four variations of Gravity, three Intervening Opportunities, and Uniform).
  - `run_model`: Converts probabilities into flow counts using constrained modeling approaches (Unconstrained, Production/Attraction Constrained, Doubly Constrained).
  - `run_law_model`: A convenience wrapper executing both steps sequentially.
- **gof** Calculates six distinct goodness-of-fit measures to evaluate the accuracy of simulated matrices against observed data. Also supports multi-exponent parallelization.

Performance considerations guided much of the design. While the R version relied on Java for computational efficiency, PyTDLM is written entirely in Python. To maintain competitive performance, the implementation makes extensive use of NumPy broadcasting, vectorization, and shared-memory multiprocessing. Parallel tasks are dispatched through `multiprocessing` pools, with data exchanged via `shared_memory` blocks, which mitigate the overhead associated with `spawn()`-based process creation on Windows and macOS and reduce copy-on-write penalties on Linux.

Benchmarks

Validation To validate the Python implementation, we performed a systematic comparison against the original Java implementation. We reproduced some case studies presented in Lenormand et al. (2016). Figure 1 (right) displays the PyTDLM results, while Figure 1 (left) shows the original results. Across several countries, laws, models, and goodness-of-fit metrics, the results were consistent between the two implementations.

DRAFT

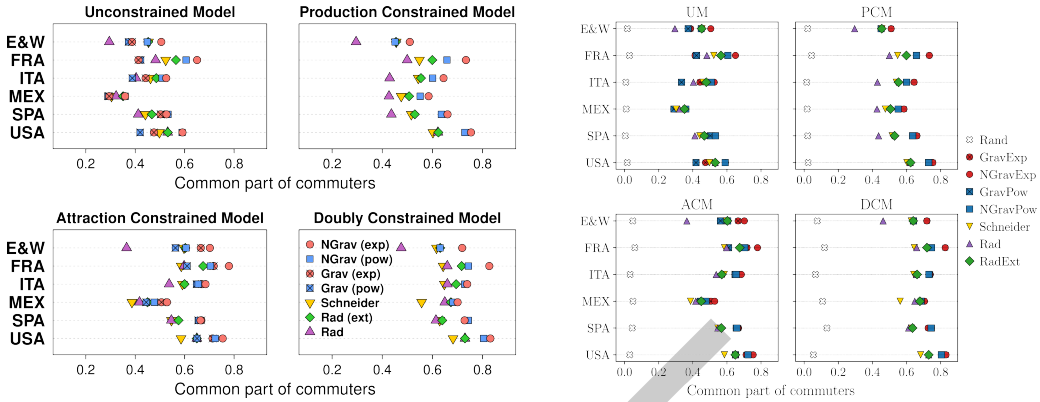

Figure 1: Common part of commuters according to the unconstrained models, the gravity and intervening opportunities laws for six case studies. Left: Java implementation, modified from Fig3 Lenormand et al. (2016), with permission of the authors. Right: PyTDLM.

Performance We benchmarked the wall-clock execution time of both packages using the example based on commuting data from Kansas in the United States in 2000. Tests were conducted on an Ubuntu 24.03.1 system equipped with a 2024 18-thread @4.6GHz CPU. As shown in Figure 2, PyTDLM demonstrates competitive performance, benefiting from vectorization and effective parallelization strategies.

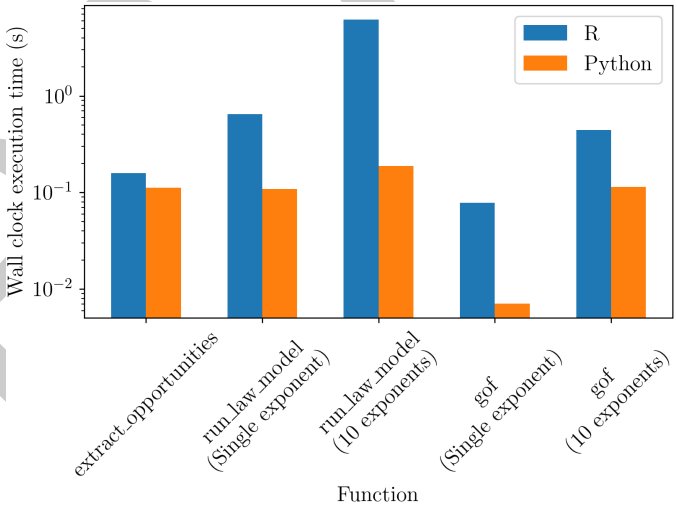

Figure 2: Comparison of R TDLM vs PyTDLM wall clock execution time, in seconds. Measurements were averaged over 100 executions of each function. `run_law_model` was run with the following parameters: `law="NGravExp", model="DCM", repli=10, return_proba=True`.

Acknowledgements

This work was supported by the Institute of Computing and Data Sciences (Project ESTIMIA) and the French National Research Agency (Project MOSIS, ANR-24-CE38-5528). The validation simulations were performed on the SACADO MeSU platform at Sorbonne Université.

References

Barbosa, H., Barthelemy, M., Ghoshal, G., James, C. R., Lenormand, M., Louail, T., Menezes, R., Ramasco, J. J., Simini, F., & Tomasini, M. (2018). Human mobility: Models and applications. *Physics Reports*, *734*, 1–74. https://doi.org/10.1016/j.physrep.2018.01.001

Dargel, L., & Laurent, T. (2021). *Spflow: Spatial econometric interaction models*. https://CRAN.R-project.org/package=spflow

Giles, J. (2021). *COVID-19-Mobility-Data-Network/mobility: mobility version 0.6.4* (Version v0.6.4). Zenodo. https://doi.org/10.5281/zenodo.5171373

Lenormand, M. (2023). TDLM: An r package for a systematic comparison of trip distribution laws and models. *Journal of Open Source Software*, *8*(88), 5434. https://doi.org/10.21105/joss.05434

Lenormand, M., Bassolas, A., & Ramasco, J. J. (2016). Systematic comparison of trip distribution laws and models. *Journal of Transport Geography*, *51*, 158–169. https://doi.org/10.1016/j.jtrangeo.2015.12.008

Lenormand, M., Huet, S., Gargiulo, F., & Deffuant, G. (2012). A Universal Model of Commuting Networks. *PLoS ONE*, *7*, e45985. https://doi.org/10.1371/journal.pone.0045985

Lovelace, R., & Nowosad, J. (2023). *Simodels: Flexible framework for developing spatial interaction models*.

Masucci, A., Serras, J., Johansson, A., & Batty, M. (2013). Gravity versus radiation models: On the importance of scale and heterogeneity in commuting flows. *Phys. Rev. E*, *88*, 022812. https://doi.org/10.1103/physreve.88.022812

Oshan, T. M. (2016). A primer for working with the spatial interaction modeling (SpInt) module in the python spatial analysis library (PySAL). *REGION*, *3*(2), R11–R23. https://doi.org/10.18335/region.v3i2.175

Pappalardo, S., L., & Pellungrini, R. (2022). Scikit-mobility: A python library for the analysis, generation, and risk assessment of mobility data. *Journal of Statistical Software*, *103*(4), 1–38. https://doi.org/10.18637/jss.v103.i04

Simini, F., González, M. C., Maritan, A., & Barabasi, A.-L. (2012). A universal model for mobility and migration patterns. *Nature*, *484*, 96–100. https://doi.org/10.1038/nature10856

Wölwer, A.-L., Burgard, J. P., Kunst, J., & Vargas, M. (2018). Gravity: Estimation methods for gravity models in R. *Journal of Open Source Software*, *3*(31), 1038. https://doi.org/10.21105/joss.01038

Yang, Y., Herrera, C., Eagle, N., & González, M. C. (2014). Limits of predictability in commuting flows in the absence of data for calibration. *Scientific Reports*, *4*(5662), 5662. https://doi.org/10.1038/srep05662

DRAFT